

EXPERIMENT-35

Algorithm:

1. Start the program.
2. Input the order n of the square matrix.
3. Check that n is positive.
4. Input all $n \times n$ elements of the matrix.
5. Repeatedly display the menu:
 - 1 – Check whether main diagonal elements are distinct.
 - 2 – Find the sum of main diagonal elements.
 - 3 – Display elements using diagonal traversal.
 - 4 – Exit.
6. Use separate functions for each operation.
7. For distinct diagonal elements, compare every $a[i][i]$ with the diagonal elements after it.
8. For the sum, add all $a[i][i]$.
9. For diagonal traversal, process diagonals according to $\text{row} + \text{column} = d$, where d goes from 0 to $2n-2$. Traverse alternate diagonals in opposite directions.
10. Stop when the user selects Exit.

Test-Case Table:

TC	Matrix	Operation	Expected Output
1	1 2 3 / 4 5 6 / 7 8 9	Distinct	1 5 9 are distinct
2	1 2 3 / 4 5 6 / 7 8 9	Sum	15
3	1 2 3 / 4 5 6 / 7 8 9	Traversal	1 2 4 7 5 3 6 8 9
4	1 2 / 3 1	Distinct	diagonal elements are not distinct
5	5	All operations	Distinct = Yes, Sum = 5, Traversal = 5
6	Any valid matrix	Choice 7	Invalid choice

Program, Compilation and Execution:

```
GNU nano 9.0 35.c
#include <stdio.h>

void checkDistinct(int n, int a[n][n]);
void diagonalSum(int n, int a[n][n]);
void diagonalTraversal(int n, int a[n][n]);

int main()
{
    int n, i, j, choice;

    printf("Enter the order of the square matrix: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Invalid matrix size.\n");
        return 0;
    }

    int a[n][n];

    printf("Enter %d elements of the matrix:\n", n * n);

    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }

    do
    {
        printf("\n—— MENU ——\n");
        printf("1. Check whether main diagonal elements are distinct\n");
        printf("2. Find sum of main diagonal elements\n");
        printf("3. Diagonal traversal\n");
        printf("4. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);
    }
```

```
GNU nano 9.0 35.c
switch (choice)
{
    case 1:
        checkDistinct(n, a);
        break;

    case 2:
        diagonalSum(n, a);
        break;

    case 3:
        diagonalTraversal(n, a);
        break;

    case 4:
        printf("Exiting program...\n");
        break;

    default:
        printf("Invalid choice. Please enter 1 to 4.\n");
}

} while (choice != 4);

return 0;
}

/* Function to check whether main diagonal elements are distinct */
void checkDistinct(int n, int a[n][n])
{
    int i, j;
    int distinct = 1;

    printf("Main diagonal elements: ");

    for (i = 0; i < n; i++)
    {
        printf("%d ", a[i][i]);
    }

    for (i = 0; i < n - 1; i++)
    {
        for (j = i + 1; j < n; j++)
        {

```

```
GNU nano 9.0 35.c
            if (a[i][i] == a[j][j])
            {
                distinct = 0;
                break;
            }
        }
    }

    if (distinct == 0)
        break;

    if (distinct)
        printf("\nAll the main diagonal elements are distinct.\n");
    else
        printf("\nAll the main diagonal elements are not distinct.\n");
}

/* Function to find sum of main diagonal elements */
void diagonalSum(int n, int a[n][n])
{
    int i;
    int sum = 0;

    for (i = 0; i < n; i++)
    {
        sum = sum + a[i][i];
    }

    printf("Sum of main diagonal elements: %d\n", sum);
}

/* Function for diagonal traversal */
void diagonalTraversal(int n, int a[n][n])
{
    int d, row, col;

    printf("Diagonal traversal: ");

    for (d = 0; d <= 2 * n - 2; d++)
    {
        if (d % 2 == 0)
        {

```

```

    }
    /* Downward-left direction */
    if (d < n)
        col = d;
    else
        col = n - 1;

    row = d - col;

    while (row < n && col ≥ 0)
    {
        printf("%d ", a[row][col]);
        row++;
        col--;
    }
}
else
{
    /* Upward-right direction */
    if (d < n)
        row = d;
    else
        row = n - 1;

    col = d - row;

    while (row ≥ 0 && col < n)
    {
        printf("%d ", a[row][col]);
        row--;
        col++;
    }
}

printf("\n");
}
}

```

```

—— MENU ——
1. Check whether main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 1
Main diagonal elements: 1 4
All the main diagonal elements are distinct.

—— MENU ——
1. Check whether main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 2
Sum of main diagonal elements: 5

—— MENU ——
1. Check whether main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 3
Diagonal traversal: 1 3 2 4

—— MENU ——
1. Check whether main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: █

```

NAME: TAANISH AGARWAL
BATCH: B-15
SAP ID: 590034662